Cadre théorique de Flask

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Le Framework Flask **3**

A. Le Framework Flask 3

B. Exercice : Quiz 5

III. Réaliser un site web grâce à Flask **6**

A. Une page web en quelques lignes 6

B. Un serveur Web avec Flask 7

C. Exercice : Quiz 9

IV. L'essentiel **10**

V. Auto-évaluation **10**

A. Exercice 10

B. Test 11

Solutions des exercices **11**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+ PyCharm Community

Pré requis : bases en Python

Contexte

Flask est ce qu'on appelle un Framework. Un Framework, c'est un environnement applicatif dont l'objectif premier est la simplification.

L'objectif de Flask, c'est de simplifier le développement d'applications Web de manière générale, qu'il s'agisse de sites web, serveurs, API Rest, etc.

Sa particularité, c'est qu'il est extrêmement simple et léger. Il permet de développer des applications web avec très peu de code. A ce titre, on dit parfois que Flask est un « *micro-framework* ».

De manière générale, Flask est avec Django, un des meilleurs et un des Frameworks les plus réputés en Python dans le domaine du développement Web.

II. Le Framework Flask

A. Le Framework Flask

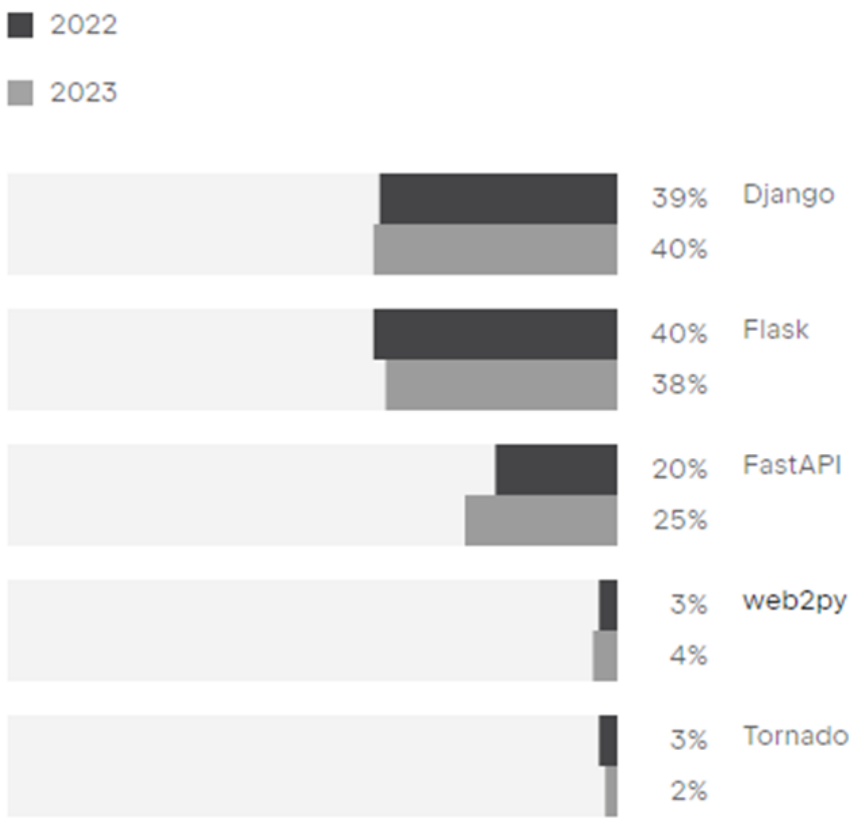
Flask est un « *micro-framework* ». Cette appellation vient du fait que Flask comporte un noyau extrêmement simplifié, de sorte que lorsque vous développez votre application, nous n'embarquez pas d'éléments inutiles. Flask permet l'utilisation d'extensions, qui vous permettront de réaliser énormément de choses, mais de manière très contrôlée. Par exemple, ces extensions vous permettront d'utiliser des bases de données, mettre en place des serveurs web, et plein d'autres choses.

Flask a été créé en 2004, sous l'impulsion d'Armin Ronacher. Il semblerait que Flask ait été créé suite à un poisson d'avril, mais il est devenu tellement efficace et populaire, qu'il en est devenu une application extrêmement populaire et développée.

Source : Flask web development, one drop at a time[1]

1 https://flask-fr.readthedocs.io/index.html

Évidemment, les Frameworks de développement Web sont nombreux en Python. Une étude récente, réalisée par la société JetBrains montre que Django et Flask sont les deux frameworks les plus utilisés dans le domaine du web-développement en Python :



Source : JETVRAINS[1]

Le choix du framework Flask s'imposera d'avantage dans les cas suivants :

Remarque

Flask, comme à peu près tous les Frameworks de développement Web en Python, sont plutôt classifiés comme Frameworks 'backend'. Contrairement au 'frontend', le 'backend' désigne le coté 'serveur' d'une application, c'est-à-dire le 'moteur' de notre site web. C'est ce 'moteur' qui traitera, par exemple :

- Les requêtes envoyées par les visiteurs,
- Les réponses à envoyer, et la génération du contenu des pages web,
- Les éventuels traitements à réaliser côté serveur,
- Les connections éventuelles vers d'autres services (cloud, base de données, etc.).

Lorsque vous souhaitez réaliser un site web complet, avec une interface graphique agréable, il vous faudra associer à votre développement Flask un développement 'frontend', c'est-à-dire le design visuel de votre site web. Pour cela, il existe également des frameworks. Citons par exemple Bootstrap (framework HTML), ou encore ReactJS (framework JavaScript), qui sont spécialisés dans le développement Frontend et qui peuvent venir se 'greffer' sur votre backend, qu'il soit développé en Flask, Django, ou n'importe quel Framework backend.

1 https://www.jetbrains.com/lp/devecosystem-2023/python/#python_web_libs_two_years

Tous droits réservés à STUDI - Reproduction interdite

Il existe de nombreux exemples d'applications Flask sur le web. Ainsi, vous trouverez une liste d'exemples au lien suivant : Python Flask Projects with Source Code (Beginners to Advanced)[1].

Parmi ces exemples, vous trouverez :

- Des systèmes de login et authentification,
- Une application 'todo list' implémentée avec Flask,
- Une application Portfolio.

Mais aussi plein d'autres exemples tout aussi intéressants les uns que les autres.

B. Exercice : Quiz

[solution n°1 p.13]

Question 1

Flask permet de …

- ○ Faire de l'analyse de données (data analysis)
- ○ Développer des applications frontend
- ○ Réaliser des applications web backend

Question 2

Flask est relativement peu utilisé.

- ○ Vrai
- ○ Faux

Question 3

Flask se démarque de par sa :

- ○ Légèreté
- ○ Lourdeur
- ○ Complexité

Question 4

Pour développer un site web avec Flask, vous pouvez utiliser n'importe quel framework Frontend Web.

- ○ Vrai
- ○ Faux

Question 5

Au départ, Flask n'avait pas pour vocation à devenir un Framework populaire.

- ○ Vrai
- ○ Faux

1 https://www.geeksforgeeks.org/flask-projects/

Tous droits réservés à STUDI - Reproduction interdite

III. Réaliser un site web grâce à Flask

A. Une page web en quelques lignes

Pour développer une application Flask, vous devez d'abord importer la librairie Flask, comme d'habitude :

```
1 from flask import Flask
```

Rentrons tout de suite dans le vif du sujet avec un petit exemple très simple :

Exemple

Nous allons réaliser un exemple très simple : une application web qui affichera uniquement le texte « *Hello, World !* » lorsqu'on s'y connectera à l'aide de notre navigateur.

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def hello_world():
7     return "<p>Hello, World!</p>"
```

Dans ce code :

- On commence par importer Flask.
- On instancie notre application en appelant Flask().
- On crée une 'route' qui 'capturera' tous les accès vers le chemin « / ». C'est le chemin qui correspond à la 'racine' de notre site web.
- On définit une fonction hello_world(), qui renvoie un contenu HTML très simple, compris entre les balises … , qui permettent de définir un paragraphe de texte.

Enregistrez votre fichier sous le nom « *main.py* ». Ensuite, dans votre terminal, tapez les deux commandes suivantes :

```
> $env:FLASK_APP = "main"
> flask run
```

La première commande permet de définir « *main* »(.py) comme votre application Flask par défaut, ce qui nous permettra de lancer notre application simplement avec la seconde commande (flask run). Cette seconde commande démarrer le serveur Flask (serveur de développement).

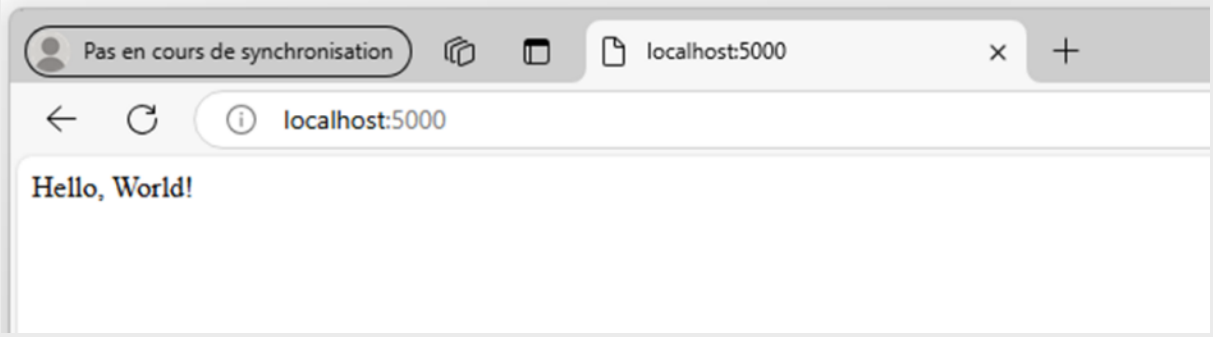
Votre IDE devrait vous renvoyer le message suivant :

```
(studi) ~\PycharmProjects\STUDI-flask\pythonProject
flask run
 * Serving Flask app 'main'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [12/Oct/2024 23:47:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Oct/2024 23:47:21] "GET /favicon.ico HTTP/1.1" 404 -
```

Ce qui signifie que votre application est en cours d'exécution. Pour la tester, rendez-vous sur le lien spécifié dans votre terminal : http://127.0.0.1:5000[1], qui correspond à l'adresse locale de votre serveur Flask.

1 http://127.0.0.1:5000/

Tous droits réservés à STUDI - Reproduction interdite



Rendu de notre application Flask

Nous avons créé notre premier serveur Web à l'aide de Flask. Comme vous pouvez le voir, c'est extrêmement simple !

B. Un serveur Web avec Flask

Nous allons développer un serveur Web minimal, qui répond tout simplement à une requête de type 'POST' et renvoie une réponse à l'utilisateur. Voyons ceci dans l'exemple suivant :

Exemple

Pour cet exemple, nous aurons besoin de réaliser une page d'accueil pour envoyer notre requête (POST) vers notre serveur Flask.

C'est là que normalement, vous pourriez utiliser un framework de développement frontend comme Bootstrap ou ReactJS. Dans notre cas, nous n'avons pas besoin d'une belle page web, nous allons nous limiter à écrire directement nous même notre page HTML.

Dans le répertoire de travail, créez à l'aide de PyCharm un nouveau fichier et nommez-le « *index.html* ». Mettez-y le contenu suivant :

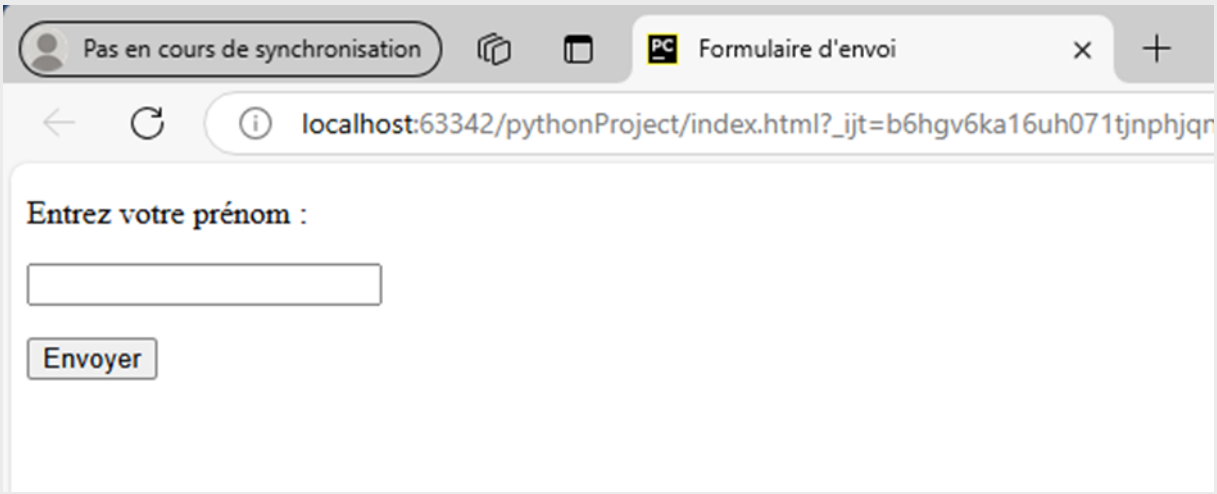
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Formulaire d'envoi</title>
6 </head>
7 <body>
8 <form action = "http://localhost:5000/login" method = "post">
9         <p>Entrez votre prénom :</p>
10        <p><input type = "text" name = "prenom" /></p>
11        <p><input type = "submit" value = "Envoyer" /></p>
12      </form>
13 </body>
14 </html>
```

L'objet de ce cours n'est pas le langage html, cependant, ce code est très simple. On définit un formulaire « *form* », dont l'action est d'envoyer une requête http de type POST à l'adresse http://localhost:5000/login.

Le contenu de la requête contiendra notamment le champ « *prenom* » qui contiendra donc le texte entré dans le champ texte par l'utilisateur.

Tous droits réservés à STUDI - Reproduction interdite

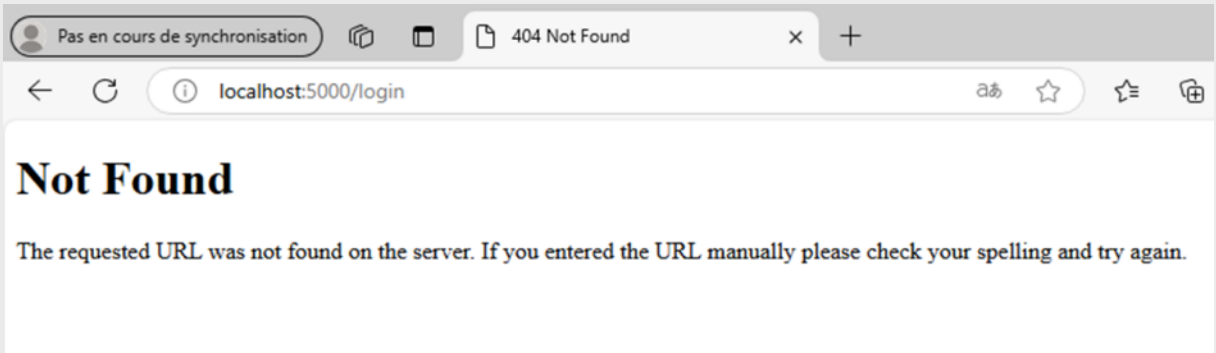
Enregistrez ce fichier, et exécutez le. Vous devriez obtenir le résultat suivant :



Page web exemple de notre formulaire

Passons maintenant à la partie serveur.

Pour l'instant, si vous tapez n'importe quoi dans le champ 'prénom' et que vous cliquez sur 'envoyer', vous recevez une erreur :



Le formulaire renvoie une erreur Not Found

Cette erreur est évidemment attendue, puisque pour l'instant, notre serveur Flask ne traite pas cette requête.

D'ailleurs, dans la console PyCharm, si votre serveur Flask est en route, vous devriez voir le message suivant :

127.0.0.1 - - [13/Oct/2024 17:30:21] "POST /login HTTP/1.1" 404 –

Ceci signifie que votre serveur Flask a reçu une requête POST sur l'adresse '/login' depuis l'adresse 127.0.0.1 (adresse locale). Le serveur a renvoyé un code d'erreur 404 (ce qui correspond bien au message ('Not Found') affiché sur notre navigateur.

Nous allons donc implémenter ce qu'il faut au niveau du serveur Flask. Dans votre fichier 'main.py', entrez le code suivant :

```
1 from flask import Flask, request, url_for, redirect
2
3 app = Flask(__name__)
4
5 @app.route("/")
```


Tous droits réservés à STUDI - Reproduction interdite

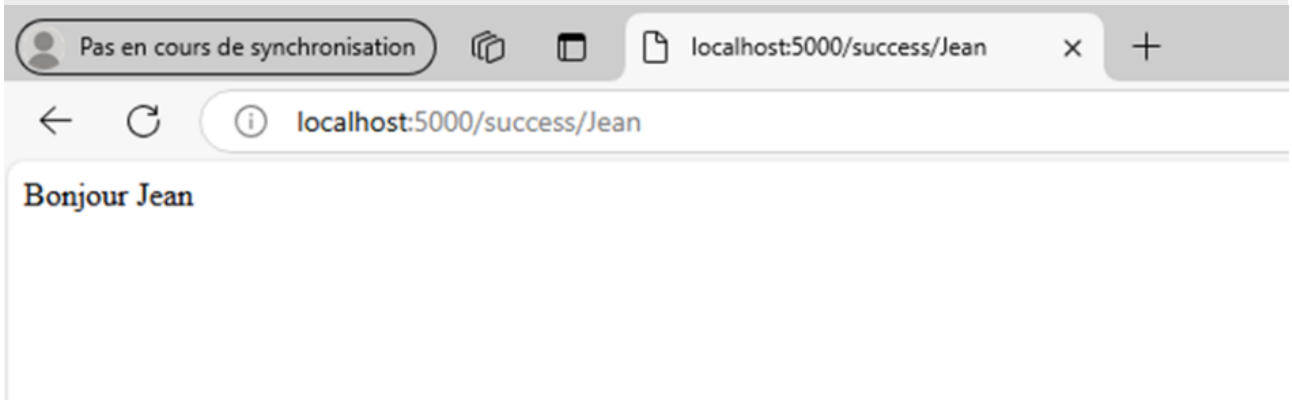
```
6 def hello_world():
7   return "<p>Hello, World!</p>"
8 @app.route('/login', methods=['POST', 'GET'])
9 def login():
10     if request.method == "POST":
11         user = request.form["prenom"]
12         return redirect(url_for('success', name=user))
13
14 @app.route('/success/<name>')
15 def success(name):
16     return 'Bonjour %s' % name
```

Dans ce code, nous importons plusieurs modules du Framework Flask :

- `Request` : permet de traiter les requêtes (GET, POST, etc.) propres aux serveurs http.
- `Url_for` : permet de trouver l'URL en fonction de la méthode qu'elle renvoie. Par exemple, nous pouvons taper url_for('success', name=user) pour appeler l'URL qui implémente la fonction 'success(name)'.
- `Redirect` : permet de gérer les routes et de rediriger l'utilisateur vers une page particulière.

Comme nous l'avons vu dans notre code html, la requête POST sera envoyée sur l'URL '/login'. Nous avons ici implémenté une méthode 'login()' qui sera exécutée lors de la réception de cette commande POST. Le contenu du formulaire envoyé via la page HTML est accessible via request.

Nous récupérons le contenu du texte entré dans le champ 'prenom' et le stockons dans la variable 'user'. Ensuite, nous redirigeons l'utilisateur vers la page l'adresse '/success/<name>' pour y afficher le mot de bienvenue :



Résultat de l'envoi d'un formulaire

C. Exercice : Quiz

[solution n°2 p.14]

Question 1

Flask permet de récupérer les requêtes http et leurs contenus.

- ○ Vrai
- ○ Faux

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Pour démarrer le serveur Flask, il faut taper :

- flask run
- flask startup
- run flask

Question 3

Pour récupérer le contenu des formulaires envoyés via POST, il faut le module :

- request
- http
- post_module

Question 4

En HTML, quelle est la bonne syntaxe pour définir un champ texte qui a le nom « *text_field* » ?

- <input type = "text" name = "text_field" />
- <input type = "text_field" name = "text" />
- <input type = "form" name = "text_field" />

Question 5

Comment définir une route (url) dans l'application Flask ?

- @app.url()
- @app.route()
- @route()

IV. L'essentiel

Flask est un incontournable du paysage de développement Web en Python. Avec Django, il constitue l'un des Frameworks les plus connu et les plus employés du moment. Sa légèreté, ses extensions ainsi que sa communauté active justifient sa popularité.

Dans ce cours, après avoir vu quelques notions théoriques de base, nous avons vu comment mettre en place une application Flask. nous avons vu comment Flask pouvait notamment être utilisé pour implémenter une application web simple, en renvoyant du contenu html, ou encore en traitant le contenu d'un formulaire envoyé.

Évidemment, les possibilités sont énormes, et nous ne pouvons pas tout couvrir à travers ce cours. Cependant, vous trouverez de nombreux exemples de toutes sortes d'applications, connues ou moins connues, réalisées avec le Framework Flask.

Dans un prochain cours, nous verrons d'ailleurs comment Flask peut être utilisé pour réaliser un serveur API Rest, ce qui encore une fois, constitue une application extrêmement utile pour tout type d'application.

V. Auto-évaluation

A. Exercice

Question 1

[solution n°3 p.15]

Créez la page HTML qui contiendra un champ de recherche ainsi qu'un bouton « *Rechercher* ».

Tous droits réservés à STUDI - Reproduction interdite

Question 2

[solution n°4 p.15]

Implémentez la partie serveur Flask correspondante.

B. Test

Exercice 1 : Quiz

[solution n°5 p.15]

Question 1

Qu'utilisez-vous pour obtenir le contenu d'un champ envoyé par formulaire ?

- request.result()
- request.form()
- request.input()

Question 2

A quoi sert la librairie 'url_for' de Flask ?

- A verifier les adresses URL
- A récupérer les routes via les méthodes qu'elle implémente
- A faire une boucle 'for' sur des URLs

Question 3

Que signifie la ligne de log Flask suivante :

127.0.0.1 - - [13/Oct/2024 17:30:21] "POST /login HTTP/1.1" 404 –

- La requête POST a été effectuée correctement
- Une erreur s'est produite lors de la réception de la requête POST
- Le serveur est en attente d'une requête POST

Question 4

Il est possible d'intégrer une bases de données avec Flask.

- Vrai
- Faux

Question 5

Flask permet d'implémenter un service API Rest.

- Vrai
- Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 5 Solution n°1

Question 1

Flask permet de …

- ○ Faire de l'analyse de données (data analysis)
- ○ Développer des applications frontend
- ◉ Réaliser des applications web backend

Flask est un Framework qui permet de développer des applications Web backend.

Question 2

Flask est relativement peu utilisé.

- ○ Vrai
- ◉ Faux

C'est faux, selon des sondages récents, Flask constitue, avec Django, un des deux Frameworks Web le plus souvent employé.

Question 3

Flask se démarque de par sa :

- ◉ Légèreté
- ○ Lourdeur
- ○ Complexité

Flask est particulièrement léger. C'est d'ailleurs ce qu'on appelle un 'micro-framework'.

Question 4

Pour développer un site web avec Flask, vous pouvez utiliser n'importe quel framework Frontend Web.

- ◉ Vrai
- ○ Faux

Flask ne contraint absolument pas le framework frontend à utiliser. Vous pouvez donc utiliser le framework que vous désirez.

Question 5

Au départ, Flask n'avait pas pour vocation à devenir un Framework populaire.

- ◉ Vrai
- ○ Faux

Vrai, Flask a été développé sans aucune vocation à être populaire, comme il l'est actuellement.

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 9 Solution n°2

Question 1

Flask permet de récupérer les requêtes http et leurs contenus.

- (●) Vrai
- ( ) Faux

Bien sûr, et c'est un élément fondamental de Flask, en tant que serveur backend.

Question 2

Pour démarrer le serveur Flask, il faut taper :

- (●) flask run
- ( ) flask startup
- ( ) run flask

C'est la première proposition qui est la bonne. flask run permet de démarrer le serveur de développement par défaut.

Question 3

Pour récupérer le contenu des formulaires envoyés via POST, il faut le module :

- (●) request
- ( ) http
- ( ) post_module

Il faut importer le module request de la librairie Flask. Celui-ci permet de récupérer les éléments envoyés dans les requêtes POST pour pouvoir les traiter coté serveur.

Question 4

En HTML, quelle est la bonne syntaxe pour définir un champ texte qui a le nom « *text_field* » ?

- (●) <input type = "text" name = "text_field" />
- ( ) <input type = "text_field" name = "text" />
- ( ) <input type = "form" name = "text_field" />

Il s'agit de la première proposition. Le type d'entrée est 'text', et le nom est 'text_field'.

Question 5

Comment définir une route (url) dans l'application Flask ?

- ( ) @app.url()
- (●) @app.route()
- ( ) @route()

Tous droits réservés à STUDI - Reproduction interdite

Il faut utiliser @app.route() et y spécifier le chemin de la route (url).

p. 10 Solution n°3

La page HTML est très similaire à ce que nous avons vu dans l'exemple du cours. Dans ce cas, la requête est toujours de type 'POST', mais elle est réalisée sur l'url '/search' de notre serveur :

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Formulaire de recherche</title>
6 </head>
7 <body>
8 <form action = "http://localhost:5000/search" method = "POST">
9         <p>Que voulez-vous chercher ?</p>
10        <p><input type="text" name="search_input" /></p>
11        <p><input type="submit" value="Rechercher" /></p>
12      </form>
13 </body>
14 </html>
```

On voit que le champ de recherche est appelé 'search_input'. C'est de nom que l'on devra utiliser dans notre application Flask pour récupérer l'entrée de l'utilisateur.

p. 11 Solution n°4

Le code Python ressemble également à ce qui a été fait durant le cours :

```
1 from flask import Flask, request, redirect
2
3 app = Flask(__name__)
4
5 @app.route('/search', methods=['POST', 'GET'])
6 def search():
7     if request.method == "POST":
8         input = request.form["search_input"]
9         return redirect(f'https://www.google.com/search?q={input}')
```

Nous n'avons qu'une seule route qui est définie, et qui correspond à l'url de notre requête POST ('/search'). Nous récupérons le contenu du champ de recherche (« *search_input* »), et nous redirigeons simplement vers la page google en ajoutant le texte correspondant à la recherche souhaitée.

Exercice p. 11 Solution n°5

Question 1

Qu'utilisez-vous pour obtenir le contenu d'un champ envoyé par formulaire ?

- ○ request.result()
- ◉ request.form()
- ○ request.input()

Tous droits réservés à STUDI - Reproduction interdite

C'est request.form() qui permet d'accéder au contenu d'un élément constitutif du formulaire envoyé dans la requete POST.

Question 2

A quoi sert la librairie 'url_for' de Flask ?

- ○ A verifier les adresses URL
- ◉ A récupérer les routes via les méthodes qu'elle implémente
- ○ A faire une boucle 'for' sur des URLs

Url_for permet de se prémunir des changements éventuels dans les routes, en se liant plutôt aux méthodes implémentées plutôt qu'aux noms des routes.

Question 3

Que signifie la ligne de log Flask suivante :

127.0.0.1 - - [13/Oct/2024 17:30:21] "POST /login HTTP/1.1" 404 –

- ○ La requête POST a été effectuée correctement
- ◉ Une erreur s'est produite lors de la réception de la requête POST
- ○ Le serveur est en attente d'une requête POST

La deuxième proposition est la bonne. Le serveur a reçu une requête POST mais celle-ci a généré une erreur 404 (not found).

Question 4

Il est possible d'intégrer une bases de données avec Flask.

- ◉ Vrai
- ○ Faux

Oui, on peut par exemple utiliser une base de données SQLite, comme nous l'avons vu précédemment. Flask n'impose aucune restriction quant au type de base de données que l'on peut utiliser.

Question 5

Flask permet d'implémenter un service API Rest.

- ◉ Vrai
- ○ Faux

Vrai, même si nous n'avons pas encore abordé son implémentation (nous le ferons dans un prochain cours), Flask permet effectivement de réaliser un service API Rest.

Tous droits réservés à STUDI - Reproduction interdite